

Entregable 2 – Concurrencia

Simulación de un restaurante · Programación Avanzada 2026 · Universidad de Montevideo
Integrantes: [Nombre Apellido], [Nombre Apellido]

1. Resumen del producto

El programa simula en Java un restaurante en el que clientes, mozos, cocineros y cajeros trabajan al mismo tiempo, cada uno en su propio hilo. La simulación corre durante un tiempo `T`. Al cumplirse, el restaurante cierra de forma ordenada: no entra nadie más, los clientes que todavía no pidieron se retiran y los que ya pidieron terminan normalmente. Cuando no queda ningún cliente, el personal termina, se liberan todos los recursos y el programa finaliza solo.

Cada cambio de estado se muestra en pantalla junto con una **foto completa del restaurante** en ese instante (mesas, clientes, mozos, cocina, cocineros y cajas) y se guarda en el archivo de texto `restaurante.log`. Este es un ejemplo real tomado del log:

```
#00159 [ 15.57s] Cajero 2 le cobra Chivito ($650) a Cliente 8
MESAS      Mesa 1: Esperando la comida [C13,C14,C15] | Mesa 2: Eligiendo menú [C16,C17,C18]
           Mesa 3: Comiendo [C7,C8,C9] | Mesa 4: Comiendo [C10,C11,C12]
CLIENTES   C7: Comiendo Pizza | C8: En la fila de la caja | C9: Comiendo Ensalada César | C10: Comiendo Pizza
           C11: Comiendo Chivito | C12: Comiendo Milanesa con fritas | C13: Pidió Ensalada César, espera la comida
           C14: Pidió Pizza, espera la comida | C15: Pidió Milanesa con fritas, espera la comida
           C16: Eligió Ñoquis con tuco, espera a su mesa | C17: Eligiendo menú (Mesa 2) | C18: Eligiendo menú (Mesa 2)
           C19: Esperando afuera
MOZOS      Mozo 1: Esperando | Mozo 2: Esperando | Mozo 3: Esperando
COCINA      Platos: vacía
COCINEROS  Cocinero 1: Esperando pedidos | Cocinero 2: Cocinando Milanesa con fritas | Cocinero 3: Cocinando Pizza
CAJAS      Fila: vacía | Cajero 1: Esperando clientes | Cajero 2: Cobrando a C8
```

Cómo ejecutar

```
cd "Entregable 2"
javac -d out src/*.java
cd out
java Main          # usa T = 30 segundos (Config.T)
java Main 60       # T = 60 segundos
```

Todas las constantes de la letra (`T`, `M`, `P`, `Z`, `C`, `Y` y los rangos de tiempo `TPmin..TPmax`, `TMmin..TMmax`, etc.) están en `Config.java`. Cada menú tiene su propio rango `TCmin..TCmax`, definido en `Menu.java`. Se necesita Java 17 o superior.

2. Diseño general

Clase	Responsabilidad
<code>Main</code>	Crea los recursos compartidos, lanza los pools de hilos, espera <code>T</code> y coordina el cierre ordenado.
<code>Restaurante</code>	Monitor del salón. Controla la puerta (capacidad <code>P*M</code>), la sala de espera donde se arman los grupos de <code>P</code> , las mesas y la lista de tareas de los mozos.
<code>Mesa</code>	Estado de la mesa, más una <code>CyclicBarrier</code> para la elección del menú y un <code>CountDownLatch</code> para el servicio.

Cliente	Tarea (Runnable) que se ejecuta en el pool de clientes.
Mozo , Cocinero , Cajero	Personal. Cada rol corre en su propio pool de tamaño fijo.
Cocina	Cola de platos por cocinar (LinkedBlockingQueue).
Cajas	Fila única de cobro (LinkedBlockingQueue).
Pedido , Plato , TareaMozo , Menu	Datos del dominio. Plato y TareaMozo son inmutables.
EstadoRestaurante	Registro del estado de todos los actores. Arma la foto de cada cambio y la envía al display.
Display	Hilo que consume la cola de notificaciones, la muestra en pantalla y la escribe en el log.
Generador	"Puerta de la calle": crea un cliente nuevo cada $TP_{min}..TP_{max}$.
Estadísticas , Azar , Config	Contadores atómicos, tiempos aleatorios y constantes.

Recorrido de un cliente

1. **Llega.** Si ya hay $P \times M$ clientes adentro, espera afuera (`wait()` en el monitor del restaurante).
2. **Entra** a la sala de espera. Cuando hay P clientes esperando y una mesa libre, el restaurante los sienta juntos. Si son menos de P , siguen esperando.
3. **Elige el menú** (al azar, demora TM) y espera en la `CyclicBarrier` de la mesa. El último en elegir ejecuta la acción de la barrera, que **llama a un mozo**.
4. **Espera al mozo.** Desde que un mozo toma la mesa, se considera que la mesa **ya realizó su pedido**.
5. **Espera la comida** en el `CountDownLatch` de la mesa. El mozo lo libera cuando sirvió los P platos, así que todos empiezan a comer juntos.
6. **Come** (TQ). Después, cada uno por su cuenta hace la **fila única de cajas** y espera a que le cobren.
7. **Se va.** Cuando se va el último de la mesa, la mesa queda sucia hasta que un mozo la limpia, y recién ahí queda libre.

Mozos, cocineros y cajeros

- **Mozos:** piden al restaurante la próxima tarea y, si no hay ninguna, esperan (`wait()`). La prioridad elegida es *servir platos listos* > *limpiar mesas* > *tomar pedidos*: primero se termina lo que ya está empezado, así la comida no se enfría y las mesas se liberan antes.
- **Cocineros:** cuando un mozo deja un pedido, sus P platos entran juntos a la cola de la cocina, en orden de llegada. Cada cocinero libre toma el primer plato sin cocinar (`take()`), así que varios cocineros pueden cocinar platos del mismo pedido en paralelo. Cada plato demora el TC de su menú. El cocinero que termina el último plato de un pedido avisa a los mozos que pueden retirarlo.
- **Cajeros:** hay una sola fila; cada cajero libre atiende al primero (`take()`) y tarda TY en cobrar.

3. Herramientas utilizadas y justificación

Usamos solo herramientas vistas en clase:

Requisito	Herramienta	Dónde y por qué
a) Exclusión mutua (al menos dos)	<code>synchronized</code>	En <code>Restaurante</code> , protege colas y estados que cambian juntos (sala de espera, mesas, tareas de los mozos). Lo elegimos porque además necesitamos <code>wait/notifyAll</code> sobre el mismo objeto. En <code>Cocina</code> , <code>recibir()</code> es <code>synchronized</code> para que los platos de un mismo pedido entren juntos a la cola.
	<code>ReentrantLock</code> (justo)	En <code>EstadoRestaurante</code> , garantiza que el cambio de estado, el armado de la foto y el encolado al display ocurran como una sola operación. Así la foto nunca queda "a medio actualizar" y el orden del log coincide con el orden real de los cambios. Al ser <i>justo</i> (fair), los hilos informan en orden de llegada y ninguno espera indefinidamente.
	Clases atómicas	En Estadísticas (<code>AtomicInteger</code> , <code>AtomicLong</code>): contadores que actualizan muchos hilos sin bloquearse. En <code>Pedido</code> , el contador de platos listos: como <code>incrementAndGet()</code> es atómico, solo el cocinero que termina el último plato avisa a los mozos (ni dos avisos ni ninguno). También generan los números de cliente y de pedido.
	<code>volatile</code>	Para banderas y campos que se leen fuera del lock que los escribe (<code>cerrado</code> , la mesa y el menú del cliente, el estado de la mesa). Garantiza que los demás hilos vean el valor actualizado y no uno guardado en caché.
b) Barrera	<code>CyclicBarrier</code>	Hay una por mesa, con <code>P</code> participantes. Los clientes esperan a que todos elijan el menú, y la acción de la barrera (que ejecuta el último en llegar) llama al mozo. Como es cíclica, se reutiliza con cada grupo que se sienta.
	<code>CountDownLatch</code>	<code>Mesa.comidaServida</code> : el mozo libera a los <code>P</code> clientes juntos cuando sirvió toda la mesa. <code>Cajas.Cobro.pagado</code> : el cajero libera al cliente cuando le termina de cobrar. En los dos casos es un evento que pasa una sola vez, que es justo lo que modela un latch.
c) Pasaje de mensajes	<code>BlockingQueue</code> (<code>LinkedBlockingQueue</code>)	Tres colas: la cola del <code>Display</code> (todas las notificaciones), la de la <code>Cocina</code> (mozos → cocineros) y la fila de <code>Cajas</code> (clientes → cajeros). <code>take()</code> deja al consumidor esperando sin gastar CPU cuando la cola está vacía, sin escribir <code>wait/notify</code> a mano.
d) Pool de hilos	<code>Executors</code>	<code>newFixedThreadPool</code> para mozos (Z), cocineros (C) y cajeros (Y). <code>newSingleThreadExecutor</code> para el display y para el generador de clientes. <code>newCachedThreadPool</code> para los clientes, que reutiliza los hilos de los que ya se fueron. El cierre se hace con <code>shutdown()</code> + <code>awaitTermination()</code> .

e) wait / notify	<code>wait()</code> / <code>notifyAll()</code>	En Restaurante esperan los clientes que están afuera, los que están en la sala, los que esperan al mozo, y también los mozos sin tareas. Siempre se usa <code>while (!condición) wait();</code> , que vuelve a verificar la condición al despertar. Usamos <code>notifyAll()</code> y no <code>notify()</code> porque en el mismo monitor esperan hilos con condiciones distintas: un <code>notify()</code> podría despertar a uno al que el cambio no le sirve, y el aviso se perdería.
------------------	--	--

Seguridad en hilos y gestión de excepciones

- Las colas que comparten varios hilos son `BlockingQueue`. Las colas internas del restaurante (`ArrayDeque`) solo se usan con el monitor tomado. Los objetos que pasan de un hilo a otro sin protección son inmutables (`Plato`, `TareaMozo`, listas de solo lectura, enums).
- `InterruptedException`: siempre se restaura la marca de interrupción (`Thread.currentThread().interrupt()`) y el hilo termina de forma ordenada.
- Los recursos se liberan en bloques `finally`; por ejemplo, el cliente siempre "sale" del restaurante si había entrado.
- `BrokenBarrierException` hace que el cliente se retire, sin colgar al resto.
- Si no se puede abrir el archivo de log, la simulación sigue mostrando todo en pantalla.
- Si un pool no termina en el tiempo máximo (no debería pasar), se fuerza con `shutdownNow()` y se informa, para que el programa nunca quede colgado.

4. Patrones de diseño concurrente

- Productor-Consumidor** (patrón principal). Aparece tres veces: mozos → *Cocina* → cocineros; clientes → *Cajas* → cajeros; todos los actores → *cola del Display* → hilo del display. Lo elegimos porque es exactamente la dinámica del restaurante: unos generan trabajo a su ritmo y otros lo consumen al suyo, y la cola los desacopla. Lo implementamos con `BlockingQueue`, la herramienta vista en clase para este patrón.
- Monitor**. `Restaurante` encapsula todo el estado del salón y solo lo expone mediante métodos `synchronized`, que esperan con `wait()` hasta que se cumpla su condición. Así toda la lógica de "¿puedo seguir?" está en un único lugar y es más fácil razonar sobre su corrección.
- Working Threads** (pool de hilos). Mozos, cocineros, cajeros y clientes se ejecutan en pools de `Executors`, que reutilizan hilos y separan la tarea (qué se hace) de su ejecución (en qué hilo corre).
- Barrier**. La `CyclicBarrier` de cada mesa sincroniza a los `P` clientes antes de llamar al mozo, y el `CountDownLatch` los libera juntos para comer.

Para terminar a los consumidores (cocineros, cajeros y display) usamos una "píldora venenosa" (`FIN`): un elemento especial que se encola al final, para que primero se procese todo lo pendiente. Así el display muestra todos los cambios antes de cerrar, como pide la letra.

5. Evitación de deadlocks

Un deadlock necesita una **espera circular** (como en el problema de los filósofos comensales). La evitamos así:

- Jerarquía de locks**: `EstadoRestaurante` es una "hoja": mientras tiene su lock nunca llama a otro objeto sincronizado. Los demás locks (`Restaurante`, `Cocina`, `Cajas`) nunca se toman uno dentro

de otro.

- El mozo arma el pedido y lo deja en la cocina **fuera** del monitor del restaurante. El cocinero avisa al restaurante que el pedido está listo sin tener ningún otro lock tomado.
- `Cocina.tomarPlato()` **no es** `synchronized`: si un cocinero esperara en `take()` con el lock tomado, ningún mozo podría dejar pedidos.
- Ningún hilo hace `sleep()` (cocinar, comer, limpiar) mientras tiene un lock tomado.

Hicimos pruebas de estrés con pocos recursos (2 mesas de 2, 1 mozo, 1 cocinero, 1 caja, un cliente cada 20–80 ms). Terminaron siempre de forma ordenada, sin deadlocks y sin necesidad de forzar el cierre.

6. Decisiones de funcionamiento

- **Capacidad:** adentro puede haber $P \times M$ clientes, contando la sala de espera y las mesas. El resto espera afuera y entra a medida que se van otros.
- **Armado de grupos:** los primeros P de la sala (en orden de llegada) se sientan en la primera mesa libre.
- **¿Cuándo "ya realizó el pedido" una mesa?** Desde que un mozo la toma de la lista de llamados (estado *Haciendo el pedido*). Ese cambio y el cierre ocurren dentro del mismo monitor, así que no hay ambigüedad: o el mozo llegó antes del cierre y la mesa sigue normalmente, o la mesa se cancela y sus clientes se retiran.
- **Menús:** hay 6 menús, cada uno con su rango de cocción y su precio. Cada cliente elige uno al azar.
- **Mesa cancelada por el cierre:** queda libre directamente, porque no hay platos que levantar. Si los clientes comieron, queda sucia y la limpia un mozo.
- **Cierre:**
 1. Se cierran las puertas y se detiene el generador (`shutdownNow` interrumpe su `sleep`).
 2. Se espera a que terminen todos los clientes; los que ya pidieron siguen con los pasos normales.
 3. Recién ahí se avisa al personal: los mozos terminan cuando no les quedan tareas (por ejemplo, limpiar) y se envían las *poison pills* a la cocina y a las cajas.
 4. El display vacía su cola y termina.
- **Display y log:** la foto se arma con el lock de `EstadoRestaurante` tomado y se encola en ese mismo momento. Por eso muestra el estado real en el instante de la notificación. El hilo del display es el único que escribe en la consola y en `restaurante.log`, así los mensajes nunca se mezclan.
- **Estadísticas finales:** clientes atendidos, clientes retirados, mesas, platos y recaudación. Sirven para verificar la consistencia; por ejemplo, la cantidad de platos cocinados coincide con la de clientes que comieron.

7. Conclusión

El producto es una simulación completa y configurable que cumple los requerimientos funcionales y técnicos, usando las herramientas de concurrencia vistas en clase. Siempre termina de forma ordenada, libera todos los recursos y deja un registro completo de lo ocurrido.

Lo más importante que aprendimos es que, en un sistema concurrente, lo difícil no es lanzar hilos sino **coordinarlos**: decidir qué estado se comparte, quién lo protege y en qué orden se toman los locks. Pensar primero el diseño (un monitor con una única responsabilidad, colas entre productores y

consumidores, una jerarquía de locks clara) evitó desde el principio la mayoría de los errores, en lugar de ir corrigiéndolos después.

También vimos que cada herramienta tiene su lugar: `CyclicBarrier` para "esperar a todos", `CountDownLatch` para eventos de una sola vez, `BlockingQueue` para pasar trabajo entre hilos y atómicos para contadores simples. Por último, el cierre ordenado fue una de las partes más delicadas: hubo que definir con precisión qué significa "ya pidió" y en qué orden se detiene cada rol, para que ningún hilo quede esperando algo que nunca va a pasar.